

MDPs, Q-Learning, Advanced Algorithms and Bellman Equations

1 Introduction to Markov Decision Processes (MDPs)

Markov Decision Processes (MDPs) formally describe an environment for Reinforcement Learning (RL). Almost all problems can be formalized as MDPs:

- Optimal control primarily deals with continuous MDPs.
- Partially observable problems can be converted into MDPs.

1.1 State Transition Matrix

For a Markov state s and successor state s' , the state transition probability is defined by:

$$P_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s]$$

The state transition matrix P defines the transition probabilities from all states s to the successor state s' :

$$P = \begin{bmatrix} P_{11} & \dots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \dots & P_{nn} \end{bmatrix}$$

where the sum of each row is exactly 1.

1.2 Markov Process & Markov Reward Process

Markov Process: A Markov Process (or Markov Chain) is a tuple (S, P) , where:

- S is a (finite) set of states.
- P is a state transition probability matrix.

Markov Reward Process: It is a Markov Chain with values. It is defined as a tuple (S, P, R, γ) , where:

- S is a (finite) set of states.
- P is a state transition probability matrix.
- R is a reward function.
- γ is the discount factor, where $\gamma \in [0, 1]$.

1.3 Return

The return is the total discounted reward from time-step t .

- The discount is the present value of future rewards.
- The value of receiving reward R after $k + 1$ steps is $\gamma^k \cdot R$.

2 Q-Learning

Q-Learning is a model-free reinforcement learning algorithm that helps an agent learn how to make the best decisions by interacting with its environment. Instead of needing a model of the environment, the agent learns purely from experience by trying different actions and seeing their results.

The core idea is that the agent builds a **Q-table** which stores Q-values. Each Q-value estimates how good it is to take a specific action in a given state in terms of the expected future rewards. Over time, the agent updates this table using the feedback it receives.

2.1 Key Components

1. **Q-Values:** Represent the expected rewards for taking an action in a specific state. These values are updated over time using the Temporal Difference (TD) update rule.
2. **Rewards:** The feedback the agent needs to evaluate its actions.
3. **Temporal Difference (TD) Update:** The agent updates Q-values using the formula:

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

Where:

- S is the current state.
 - A is the action taken by the agent.
 - S' is the next state the agent moves to.
 - A' is the best next action in state S' .
 - R is the reward received for taking action A in state S .
 - γ (Gamma) is the discount factor which balances immediate and future rewards.
 - α (Alpha) is the learning rate determining how much new information affects old Q-values.
4. **ϵ -greedy Policy (Exploration vs. Exploitation):** Helps the agent decide which action to take based on current Q-value estimates:
 - *Exploitation:* The agent picks the action with the highest Q-value with probability $1 - \epsilon$.
 - *Exploration:* With probability ϵ , the agent picks a random action, exploring new possibilities to learn.

2.2 How Does Q-Learning Work?

The general flow is: Initialize Q-table → Choose an action → Perform action → Measure reward → Update Q-table → Repeat until termination.

Step-by-step Process:

1. Start at some initial state.
2. Agent selects an action.
3. Action is executed and the environment responds with feedback.
4. Learning Algorithm updates the Q-table.
5. Policy gets refined, and the cycle repeats.

Methods of Q-value: Temporal Difference and Bellman's Equation.

What is a Q-table? It is a memory structure where the agent stores information about which actions yield the best rewards in each state.

3 Monte Carlo Methods

Monte Carlo policy evaluation is a technique that estimates the effectiveness of a policy. It does not require a pre-built model; instead, it learns from the outcomes of the episodes it experiences. Each episode consists of a sequence of states, actions, and rewards.

3.1 How Monte Carlo Policy Evaluation Works

- The method works by running episodes where an agent interacts until it reaches a terminal state.
- At the end of each episode, the algorithm looks back at the states visited and the rewards received to calculate the Return.
- It repeatedly simulates episodes, tracking the total rewards that follow each state and then calculating the average.
- These averages give an estimate of the state value.
- By aggregating the results over many episodes, the method converges to the true value of each state when following the policy.
- Over time, as the agent learns the value of different states, it can refine its policy, favoring actions that lead to higher rewards.

4 Proximal Policy Optimization (PPO)

PPO stands for Proximal Policy Optimization. It helps agents improve their actions while keeping learning stable. It directly updates the policy like other policy gradient methods but uses a clipping rule to limit large, destabilizing changes.

Parameters in PPO:

1. **Clip range (ϵ):** Controls how much the new policy can deviate from the old.
2. **Learning Rate:** Step size for updating network weights during training.
3. **Discount Factor (γ):** Determines how much future rewards are valued compared to immediate rewards.
4. **GAE Lambda (λ):** Balances bias and variance in advantage estimation using Generalized Advantage Estimation.
5. **Number of Epochs:** How many times each batch of data is used for policy updates.
6. **Batch Size:** Number of samples per update, affecting stability and efficiency.
7. **Value Loss Coefficient (c_1):** Weight given to the value function loss in the total objective.
8. **Entropy Coefficient (c_2):** Encourages exploration by penalizing overconfident, deterministic policies.

5 Deep Deterministic Policy Gradient (DDPG)

DDPG stands for Deep Deterministic Policy Gradient. It is an algorithm that simultaneously learns two things:

1. **A Q-function:** Evaluates how good a particular action is in a particular state. It is trained using off-policy data and the Bellman equation.
2. **A Policy:** Tells the agent what action to take in a given state. It uses the Q-function to learn and improve.

DDPG is mainly used when dealing with **continuous action spaces**. In standard Q-learning with discrete actions, finding the optimal action $a^*(s)$ is easy: just calculate the Q-value for each possible action and pick the one with the maximum value:

$$a^*(s) = \arg \max_a Q^*(s, a)$$

However, in a continuous action space, there are infinitely many possible actions. Because the space is continuous, the Q-function $Q^*(s, a)$ is presumed to be differentiable with respect to the action a .

- Instead of searching for the action that maximizes the Q-value, DDPG learns a deterministic policy function denoted as $\mu(s)$.
- This policy function acts as an approximator that directly outputs the specific action that should maximize the Q-value.
- Because the Q-function is differentiable, the algorithm can use a simple, efficient gradient-based rule to update the policy $\mu(s)$.

6 Soft Actor-Critic (SAC)

SAC stands for Soft Actor-Critic. Like DDPG, it is used for solving complex continuous control tasks.

In standard reinforcement learning, the agent's sole objective is to maximize the expected sum of future rewards. SAC alters this by using a **maximum entropy objective**. The agent is trained to maximize a trade-off between two things:

1. **Expected Return:** Earning as much reward as possible.
2. **Entropy:** Acting as randomly as possible while still achieving high rewards.

6.1 DDPG vs. SAC

The main difference between DDPG and SAC lies in the type of policy they learn:

- **DDPG** learns a *deterministic policy*: for a specific state s , it will always output the exact same action a .
- **SAC** learns a *stochastic policy*: it outputs a probability distribution over possible actions, promoting better exploration and robustness.

7 Bellman Equations

We know, the expected return G_t can be expressed as:

$$G_t = \underbrace{R_{t+1}}_{\text{immediate reward}} + \underbrace{\gamma}_{\text{Discount factor}} R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma G_{t+1}$$

7.1 Derivation of State-Value Function (V^π)

It is the expected return starting from state s and following policy π .

Definition: $V^\pi(s)$ is defined as the expected total return (G_t), we will get if started in state s and follow policy forever.

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \end{aligned}$$

Now, sum over all possible actions the policy could take, and all possible next states and rewards the environment could return:

$$V^\pi(s) = \underbrace{\sum_a}_{\substack{\text{iterate} \\ \text{over all} \\ \text{action space}}} \underbrace{\pi(a \mid s)}_{\substack{\text{current} \\ \text{policy}}} \underbrace{\sum_{s', r}}_{\substack{\text{next state,} \\ \text{immediate reward} \\ \text{(iterate over all} \\ \text{possible cases)}}} p(s', r \mid s, a) \left[r + \gamma \underbrace{\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']}_{V^\pi(s')} \right]$$

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')]$$

7.2 Derivation of Action-Value Function (Q^π)

It is the expected return starting from state s , taking action a , and thereafter following policy π .

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \end{aligned}$$

Here the initial action is fixed so we do not sum over all the actions. We only sum over rewards returned by environment:

$$\begin{aligned} Q^\pi(s, a) &= \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \underbrace{\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']}_{V^\pi(s')} \right] \\ Q^\pi(s, a) &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')] \end{aligned}$$

Relationship Between V and Q

$$\begin{aligned} V^\pi(s) &= \sum_a \pi(a \mid s) Q^\pi(s, a) \\ Q^\pi(s, a) &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')] \end{aligned}$$